

Advanced C Topics

Luís Lopes

Faculdade de Ciências
Universidade do Porto

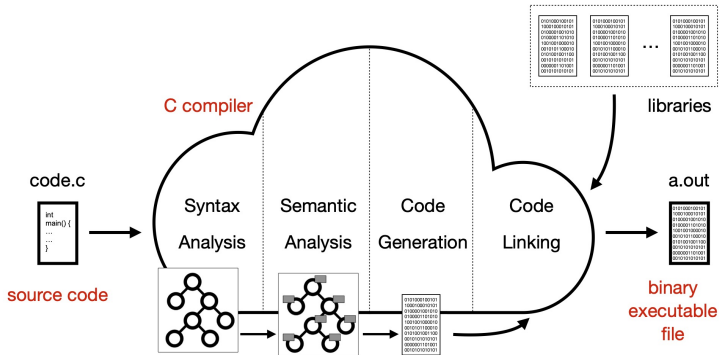
`lmlopes@fc.up.pt`

Overview

- 1 the C compiler (`gcc`)
- 2 programs and processes
- 3 stack structure
- 4 heap management
- 5 the `main()` function
- 6 datatypes
- 7 libraries

The C Compiler

- the C compiler, `gcc`, transforms a C source program
- ... say `code.c`, into an executable binary file, `a.out`
- ... in 4 major steps:



The C Compiler

Syntatic Analysis:

- checks the program for grammatical errors
- e.g., missing ';', mismatched '{}' or '()'
- e.g., using keywords for variable names as in `int if = 2`
- program text is represented in memory as a data structure
- usually some kind of tree and is called 'syntactic tree'

The C Compiler

Semantic Analysis:

- checks the syntactic tree for type errors
- e.g., `int x = "abc",`
- e.g., calling `f(5, "abc")` and `f` has type `void f(int, int)`
- this stage also gathers important information about functions
- e.g., the number of local variables and arguments
- e.g., the types of local variables and arguments
- e.g., the scopes of variables
- the syntactic tree is annotated with this information
- ... as it is crucial for the next stage

The C Compiler

Code Generation:

- traverses the syntactic tree and generates binary code
- the binary code is composed of several segments
- ... namely: `.text`, `.data` and `.bss`
- there may be also extra segments that list missing code...
- ... functions used but whose code is not written in `code.c`
- ... and symbolic information for debugging
- ... if you use option `-g`
- the compiler can be told to stop at this step
- if option `-c` is used binary file `code.o` is generated
- if option `-S` is used assembly code `code.s` is generated
- assembly code is textual representation close to binary code

The C Compiler

Linking:

- linking: completes the binary file with missing code
- checks the functions for which code is missing
- a list of such functions was built during code generation
- the compiler looks for this code in system or user libraries
- it knows the location of the Standard C library `libc`
- ... but must be guided for others using options `-L` and `-lY`
- if successful, the final executable binary file `a.out` is produced

The C compiler

- change the compiler behaviour using command line options
- you may combine options, for example:

(-Wall: show all warnings)

```
$ gcc -Wall code.c
```

(-g: add debugging information)

```
$ gcc -g code.c
```

(-c: just generate binary, not executable)

```
$ gcc -c code.c
```

(-S: generate assembly code)

```
$ gcc -S code.c
```

(-o: change name of executable file to X)

```
$ gcc code.c -o X
```

(-L -l: add code from libY located in dir X)

```
$ gcc code.c ... -LX -lY
```


The C compiler

- consider a single file C program `code.c`
- it must contain a `main()` function
- we compile it as follows:

```
$ ls
code.c
$ gcc -Wall code.c
$ ls
a.out  code.c
(a.out is default name of executable binary file)

$ ./a.out
(running program = process)
```

The C compiler

- multiple-file C program: `code1.c`, `code2.c`, ... ,`coden.c`
- must contain a single `main()` function
- compiled as follows:

```
$ ls
code1.c code2.c ... coden.c
$ gcc -Wall -c code1.c
$ gcc -Wall -c code2.c
...
$ gcc -Wall -c coden.c
$ ls
code1.c code1.o code2.c code2.o ... coden.c coden.o
```

The C compiler

Final linking and production of binary executable file:

- if we don't need extra code from libraries
- and want to keep default name for executable

```
$ gcc code1.o code2.o ... coden.o
```

```
$ ./a.out (program running = process)
```

The C compiler

Final linking and production of binary executable file:

- if we don't need extra code from libraries
- and we want to change name of executable to X

```
$ gcc code1.o code2.o ... coden.o -o X
$ ./X      (program running = process)
```

The C compiler

Final linking and production of binary executable file:

- if we need extra code from `libZ` located in directory `Y`
- and we want to change name of executable to `X`

```
$ gcc code1.o code2.o ... coden.o -o X -LY -lZ
$ ./X      (program running = process)
```

The C compiler

- C programs are collections of functions
- the entry point to the program is `main()`
- your programs will use functions that you write
- (their code is in the source files you provide)
- it also uses functions for which you do not provide code
- (e.g., `scanf()`, `printf()`, `sin()`, `exp()`, `strcmp()`, ...)
- the binary code for these latter functions will be missing
- must tell the compiler where to get that code
- the code will be kept in system libraries like `libc` and `libm`

The C compiler

- the compiler knows the location of `libc`
- this is the Standard C Library and typically includes
- ... functions like `scanf()`, `printf()` and `strcmp()`
- the location of other libraries will not be known to the compiler
- you can pass this information using the `-LX -lY` options
- ... get extra code from `libY` located in directory `X`
- sometimes, you do not know this in advance
- and the compiler may issue errors

The C compiler

- if you get an error like

`implicit declaration of function X`

- (this error occurs early, during *Semantic Analysis*)
- the compiler is telling you that you are using function `X`
- but the first time it sees it used it does not know its type
- cause: you forgot to provide the type of the function
- types for library functions are in header files, e.g., `math.h`,
- there may be one or more of these files for each system library
- run the command `man X` and check the section `Synopsis`
- include the headers therein in the source files that use `X`
- recompile afterwards

The C compiler

- if you get an error like

undefined reference to X

- (this error occurs late, during *linking*)
- the compiler is telling you that you are using function X
- but it cannot locate the binary code for the function
- without it, the final executable binary file cannot be generated
- cause: you forgot to indicate the library that contains the code
- the required code is usually stored in a system or user library
- these are named `libX`, where X is a string
- they have extensions like `.a`, `.so`, `dynlib`, ...
- e.g., `libm.so` (in this case X=m) for the math library
- run the command `man X` and check the section `Synopsis`
- add linking options therein, e.g., `-lm`, to compilation command
- recompile afterwards

Overview

- an executable binary file is a *program*
- when you run a program, it turns into a *process*

process = running program

- the same program can give rise to multiple processes

(one process)

```
$ ./a.out
```

(two processes)

```
$ ./a.out ; ./a.out
```

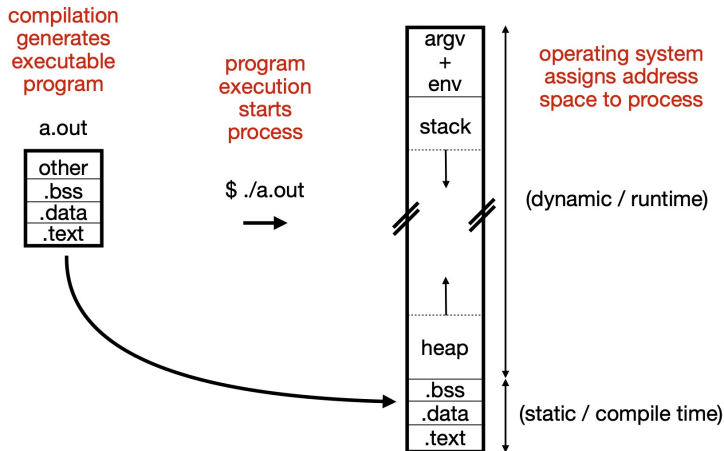
(three processes)

```
$ ./a.out ; ./a.out ; ./a.out
```

...

Process Address Space

- a (virtual) memory region exclusively used to execute a process
- operating system assigns one to each starting process



Process Address Space

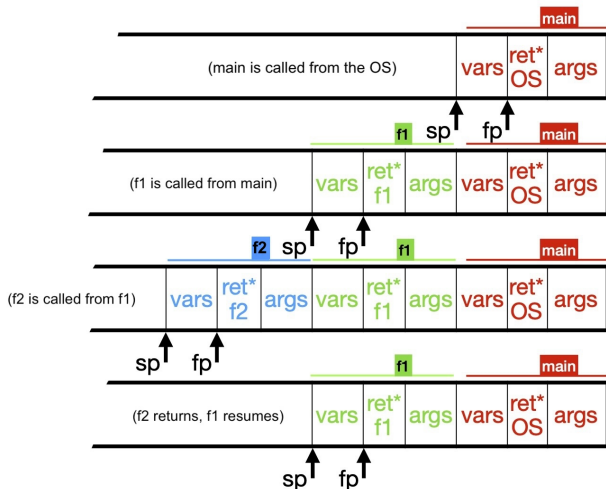
- static space: `.text`, `.data` and `.bss`
- (`.bss` = block starting symbol, uninitialized global variables)
- stores the binary code for the program and static data
- ... as copied from the program executable binary file
- dynamic space: `stack` and `heap`
- this part of the address space is used only during runtime
- stack keeps track of function calls
- data stored in the stack is ephemeral
- heap used to store longer-lived data
- variables in the stack keep pointers to heap data

Process Address Space

```
ret* f1 → int main() {  
           main vars;  
           f1(f1 args);  
           return;  
}
```

```
ret* f2 → void f1(f1 args) {  
           f1 vars;  
           f2(f2 args);  
           return;  
}
```

```
void f2(f2 args) {  
    f2 vars;  
    ...  
    return;  
}
```



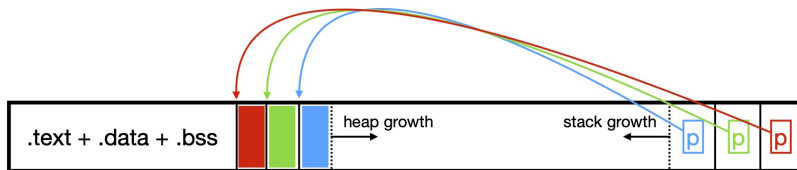
Process Address Space

- heap management done by the programmer:
- `void* malloc(size_t size)`
- `void* calloc(size_t count, size_t size)`
- `void* realloc(void *ptr, size_t size)`
- `void free(void *ptr)`
- the functions are available from `libc`
- C keeps information on allocated blocks
- is the block used? what is the size of the block?
- freed blocks are reused in further `malloc()` calls
- blocks have different sizes, hence fragmentation of heap

Process Address Space

- heap size is limited
- hence, at some point, `malloc()` may fail
- operating system may extend heap size up to a limit
- usually through system calls such as:
- `void* brk(const void* addr)` (mostly deprecated)
- `void* mmap(void* addr, ...)` (current implementations)
- the operating system limits heap growth, c.f. `getrlimit()`

Process Address Space



```
int main() {  
    char* p = (char*)malloc(16*sizeof(char));  
    f1();  
    return 0;  
}
```

```
void f1() {  
    int* p = (int*)malloc(4*sizeof(int));  
    f2();  
    return;  
}
```

what happens
to space in heap
when functions
return?

```
void f2() {  
    double* p = (double*)malloc(2*sizeof(double));  
    ...  
    return;  
}
```

why (in this
example) are the
blocks of the
same size?

Process Address Space

- some languages use Garbage Collection
- main idea: the heap is automatically managed
- no interference from programmer
- automatic identification of items no longer in use
- reuse the space freed by these items
- eventually optimize layout (e.g., compaction)
- concept attributed to John McCarthy (LISP, 1959)

Process Address Space

- why use garbage collection?
- manual heap management is prone to error:
 - e.g., memory leaks
 - e.g., memory corruption
 - it is especially hard for inexperienced programmers
- most modern programming languages use garbage collection
 - e.g., Java, Python, Haskell, Go, ...
 - older and/or low-level languages do not use garbage collection
 - e.g., besides C, C++, Objective-C, Rust

Pointer Arithmetic

consider the following program where `x`, `y`, `z`, `u`, `w` are pointers

```
#include <stdio.h>

int main( int argc, char* argv[] ) {
    char  x[4];
    int    y[4]; long z[4];
    float u[4]; double w[4];
    printf("x >> %p,%p,%p,%p\n", x, x+1, x+2, x+3);
    printf("y >> %p,%p,%p,%p\n", y, y+1, y+2, y+3);
    printf("z >> %p,%p,%p,%p\n", z, z+1, z+2, z+3);
    printf("u >> %p,%p,%p,%p\n", u, u+1, u+2, u+3);
    printf("w >> %p,%p,%p,%p\n", w, w+1, w+2, w+3);
    return 0;
}
```

Pointer Arithmetic

- the output is:

```
x >> 0x16dcdcf3d4, 0x16dcdcf3d5, 0x16dcdcf3d6, 0x16dcdcf3d7
y >> 0x16dcdcf438, 0x16dcdcf43c, 0x16dcdcf440, 0x16dcdcf444
z >> 0x16dcdcf418, 0x16dcdcf420, 0x16dcdcf428, 0x16dcdcf430
u >> 0x16dcdcf408, 0x16dcdcf40c, 0x16dcdcf410, 0x16dcdcf414
w >> 0x16dcdcf3e8, 0x16dcdcf3f0, 0x16dcdcf3f8, 0x16dcdcf400
```

- notice how each increment of a pointer of type `T`
- ... advances the address by `sizeof(T)` bytes, here: 1, 4, 8, 4, 8
- ... this is a fundamental property of pointer arithmetic in C
- `T*` types have a size equal to the addressing bits of the CPU
- ... 4 bytes (32 bit) and 8 bytes (64 bit, as in example)

Call By Value and Pointers

- consider the following example:

```
int main( int argc, char* argv[] ) {  
    int p = 2, q = 3;  
    swap(p,q);  
    printf("p = %d, q = %d\n", p, q);  
    return 0;  
}  
  
void swap( int a, int b ) {  
    int tmp; tmp = a; a = b; b = tmp;  
}
```

- function `swap()` will not do as advertized
- ... the `printf()` call in `main()` will print
- ... `p` and `q` unchanged

Pointer Arithmetic

- now we slightly rewrite it:

```
int main( int argc, char* argv[] ) {  
    int p = 2, q = 3;  
    swap(&p,&q);  
    printf("p = %d, q = %d\n", p, q);  
    return 0;  
}  
  
void swap( int* a, int* b ) {  
    int tmp; tmp = *a; *a = *b; *b = tmp;  
}
```

- this will swap the values of `p` and `q`
- important: pass a pointer for a data item to a function
- ... if you want the function to change its value

The `void` Type

- C's `void` type is used as an indicator that a function:
- ... has no return value
- ... has no arguments
- ... or both

(no return value and a character array argument)

```
void f( char* )
```

(no arguments and returns an integer)

```
int f( void )
```

(no arguments and no return value)

```
void f( void )
```

The `void` Type

- its pointer type, `void*`, is used to keep addresses for
- ... data whose type is momentarily unknown or
- ... whose type is not required (e.g., if you are copying one
- ... block of data between two memory regions)
- ... important: `void*` pointers cannot be dereferenced
- ... they must be cast to "real" types first
- the `malloc()` function from `libc` returns `void*`

```
void* p = malloc(...);  
( the following triggers an error )  
int q = *p;  
( the following triggers error: sizeof(void) = ? )  
p = p + 1;  
( the following is ok )  
int q = *((int*)p);
```


The `main()` Function

- the `main()` is the entry point to every C program
- the function is called from the operating system
- it can receive arguments provided in the command line
- it can access the program environment
- its type is `int main(int argc, char* argv[])`
- `argc` is the number of command line arguments
- `argv` is an array of string with the arguments

The `main()` Function

- consider the following program (say, `test_args.c`)
- it shows the use of `argc` and `argv`

```
int main( int argc, char* argv[] ) {  
    printf("number of arguments = %d\n", argc);  
    for( int i = 0 ; i < argc ; i++ )  
        printf("argument[%d] = %s\n", i, argv[i]);  
    return 0;  
}
```

The `main()` Function

- note that:
- the value of `argc` includes the program name
- each element of `argv` is a string

```
$ gcc -Wall test_args.c -o test_args
$ ./test_args
number of arguments = 1
argument[0] = ./test_args
$ ./test_args mercury venus earth
number of arguments = 4
argument[0] = ./test_args
argument[1] = mercury
argument[2] = venus
argument[3] = earth
```

Datatypes

- C has primitive types such as `char`, `int`, `long`, `float`, `double`
- and their pointer types `char*`, `int*`, `long*`, `float*`, `double*`
- you can define new datatypes using the `struct` construct
- for example:

```
struct {  
    char* content;  
    int   size;  
} string;
```

```
struct string s;
```

Datatypes

- can be made more concise by using the keyword `typedef`
- it creates type aliases, for example:

```
typedef X Y;  
( means type Y is an alias for type X )
```

```
typedef struct {char* content; int size;} string;
```

```
( now you can write just )  
string s;
```

```
( instead of )  
struct string s;
```

Libraries

- often it is useful to define a set of functions that
- operate on a given datatype, e.g., files, strings, numbers
- once implemented, these functions can be compiled
- ... and their binary code stored in archives called *libraries*
- libraries can be used by any application
- they simplify application development and
- ... improve stability and performance
- to implement one, we start by defining
- ... the Application Programmer's Interface (API) of the library
- = datatypes used + types for all functions

Libraries

- we sketch a library to operate on "string" type
- ... using the above definition of type `string`
- we put the datatype and API in a header file `string.h`

```
typedef struct {char* content; int size;} string;
```

```
typedef enum {false=0, true=1} boolean;
```

```
int      string_size(string*);  
char*    string_content(char*);  
string*  string_new(char*);  
string*  string_dup(string*);  
string*  string_cat(string*, string*);  
boolean  string_cmp(string*, string*);
```

Libraries

- we write each function in its own file
- e.g., here are `string_size.c` and `string_content.c`
- note that we need `#include "string.h"`
- ... so that the type `string` will be known to the compiler

```
#include "string.h"
```

```
int string_size(string* s) {  
    return s->size;  
}
```

```
#include "string.h"
```

```
char* string_content(string* s) {  
    return s->content;  
}
```


Libraries

string_new.c

```
#include "string.h"
```

```
string* string_new(char* p) {  
    string* s = (string*)malloc(sizeof(string));  
    char* q = p; s->size = 0;  
    while ( *q++ != '\0' )  
        s->size++;  
    s->content = (char*)malloc((s->size+1)*sizeof(char));  
    q = p; char* r = s->content;  
    while ( *q != '\0' )  
        *r++ = *q++;  
    *r = '\0';  
    return s;  
}
```

Libraries

```
string_dup.c
```

```
#include "string.h"
```

```
string* string_dup(string* s) {  
    string* d = (string*)malloc(sizeof(string));  
    d->size = s->size;  
    d->content = (char*)malloc((d->size+1)*sizeof(char));  
    char* p = s->content;  
    char* q = d->content;  
    while ( *p != '\0' )  
        *q++ = *p++;  
    *q = '\0';  
    return d;  
}
```

Libraries

```
string_cat.c
```

```
#include "string.h"
```

```
string* string_cat(string* s1, string* s2) {  
    string* s = (string*)malloc(sizeof(string));  
    s->size = s1->size + s2->size;  
    s->content = (char*)malloc((s->size+1)*sizeof(char));  
    char* p = s1->content; char *q = s->content;  
    while ( *p != '\0' )  
        *q++ = *p++;  
    p = s2->content;  
    while ( *p != '\0' )  
        *q++ = *p++;  
    *q = '\0'; return s;  
}
```

Libraries

`string_cmp.c` is an exercise ...

Libraries

- we are now ready to build the string library
- I will call it `libstring`
- there are two types of library:
- *static libraries* and *dynamic libraries*
- both are archives of pre-compiled binary code
- they differ in how that code is attached to the
- ... executable binary file

Libraries

- static libraries:
- the linker identifies the missing code (functions)
- consults the available libraries
- copies needed code from libraries
- attaches it to `a.out`
- updates the call addresses in `a.out`
- pros and cons:
- + faster access to code
- + executable is self-contained
- - large executables
- - redundant copies of same code in memory and disk

Libraries

- dynamic libraries:
- the linker identifies the missing code (functions)
- consults the available libraries
- for each function call to a library function
- ... inserts code that at runtime finds library and makes call
- (it does not copy the code to `a.out`)
- finding the library requires help from operating system

- pros and cons:
- - (slightly) slower access to code
- - executable requires libraries at runtime (not self-contained)
- + smaller executables
- + only one copy of library in memory shared by all processes

Libraries

- building a static library `libstring.a`
- (the extension `.a` means "archive")

```
$ ls
string.h string_new.c string_size.c
string_dup.c string_cat.c string_cmp.c
$ gcc -Wall -c string_new.c
$ gcc -Wall -c string_size.c
$ gcc -Wall -c string_content.c
$ gcc -Wall -c string_dup.c
$ gcc -Wall -c string_cat.c
$ gcc -Wall -c string_cmp.c
$ ar -rc libstring.a string_*.o
( check content of library using one of 2 commands )
$ ar -t libstring.a
( note the .o object files inside )
```


Libraries

- building a dynamic library `libstring.so`
- (the extension `.so` means "shared object")

```
$ ls
string.h string_new.c string_size.c
string_dup.c string_cat.c string_cmp.c
$ gcc -Wall -fPIC -c string_new.c
$ gcc -Wall -fPIC -c string_size.c
$ gcc -Wall -fPIC -c string_content.c
$ gcc -Wall -fPIC -c string_dup.c
$ gcc -Wall -fPIC -c string_cat.c
$ gcc -Wall -fPIC -c string_cmp.c
$ gcc -shared -o libstring.so string_*.o
$ nm libstring.so
( note the .o object files inside )
```

Libraries

- we test the library using `test_string.c`

```
#include <stdio.h>
#include "string.h"

int main( int argc, char* argv[] ) {
    string* s1 = string_new(argv[1]);
    string* s2 = string_new(argv[2]);
    string* s3 = string_dup(s1);
    string* s4 = string_cat(s1,s2);
    printf("string1 = %s\n", string_content(s1));
    printf("string2 = %s\n", string_content(s2));
    printf("string3 = %s\n", string_content(s3));
    printf("string4 = %s\n", string_content(s4));
    return 0;
}
```

Libraries

- we compile `test_string.c` by signaling its location
- ... and name to `gcc` using the command line
- ... options `-L` and `-l`

```
$ gcc -Wall test_string.c -o test_string -L. -lstring
( if libstring is dynamic run following command )
$ export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
( finally, we run the test program with
  ... some command line arguments )
$ ./test_string  fire fly
fire
fly
fire
firefly
```

Function Pointers

- in C it is possible to use pointers to functions that,
- ... as the name suggests, point to the beginning of the
- ... code of a function in the `.text` segment
- the following instruction

```
typedef void (*voidvoidfunction)( void );
```

- defines a new type `voidvoidfunction`
- ... that can be used to hold pointers to functions
- ... without return values
- ... and without arguments

Function Pointers

- using this type, we write a simple example `functions.c`

```
typedef void (*voidvoidfunction)( void );
```

```
void f1( void ) { printf("f1 called\n"); }
```

```
void f2( void ) { printf("f2 called\n"); }
```

```
void f3( void ) { printf("f3 called\n"); }
```

```
static voidvoidfunction fp[3] = { f1, f2, f3 };
```

```
int main( int argc, char* argv[] ) {  
    for ( int i = 1 ; i < argc ; i++ )  
        fp[argv[i][0] - '0' - 1]();  
    return 0;  
}
```

Function Pointers

- compiling and executing as follows we get:

```
$ gcc -Wall functions.c -o functions
$ ./functions 1 2 2 3 1 2
f1 called
f2 called
f2 called
f3 called
f1 called
f2 called
```